Artificial Intelligence

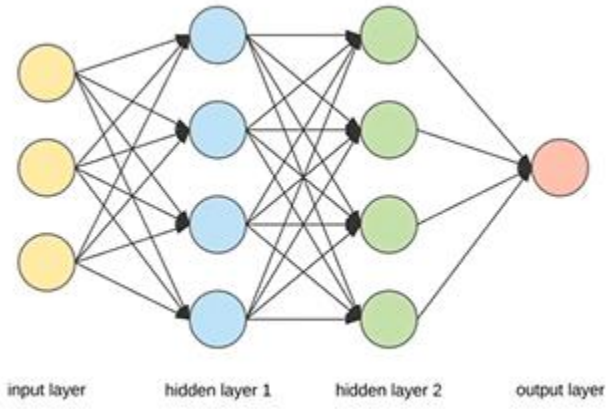
Artificial Intelligence is a branch of Computer Science that is concerned with building smart and **intelligence machines**. Intelligence machine can think and make decision on their own. Such as Self driving car, Google assistant etc.

Machine learning:

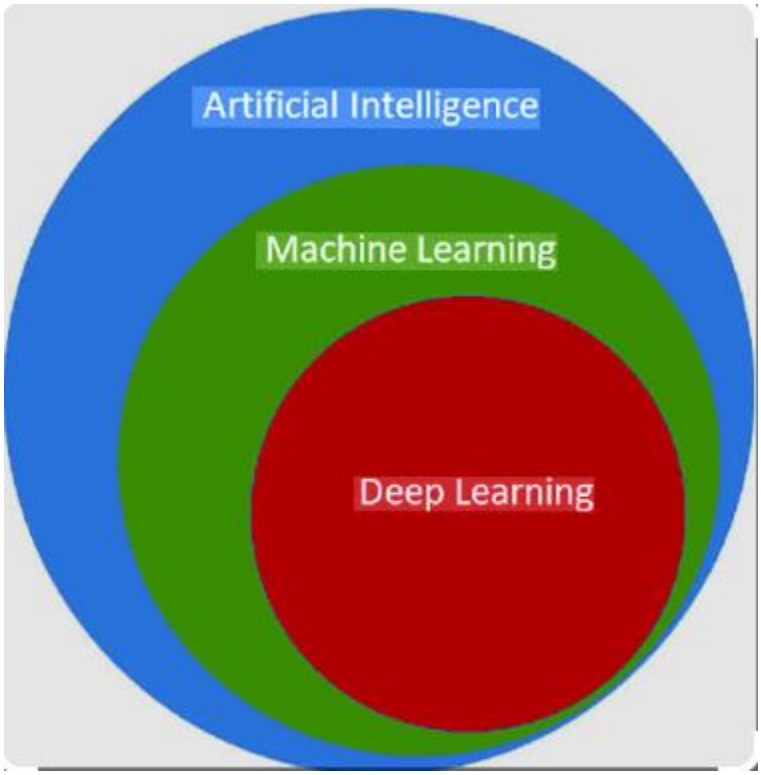
Machine learning is a technique to implement AI that can learn from the data by themselves without being explicitly programmed. Examples Cat or Dog recognition providing large number of images of car and dog.

Deep learning:

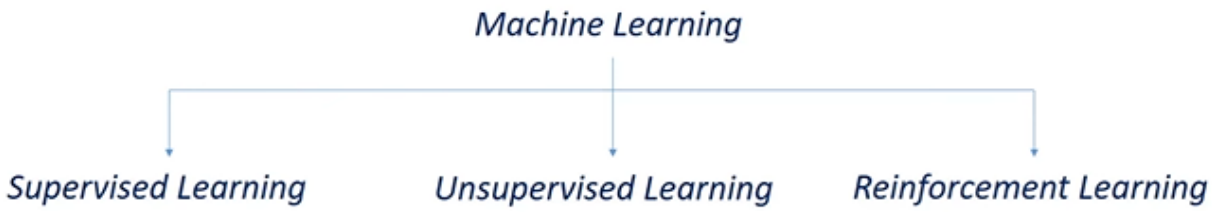
Deep learning is a subfield of Machine learning that uses Artificial Neural Networks (Mathematical models that are connected to each other as like neurons in human brain) to learn from the data. There are different layers in those artificial neural networks.



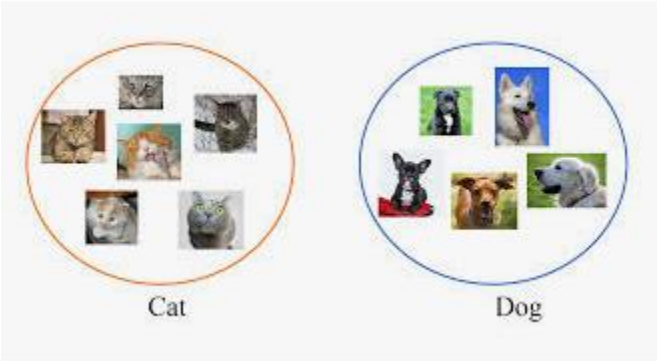
AI vs ML vs DL



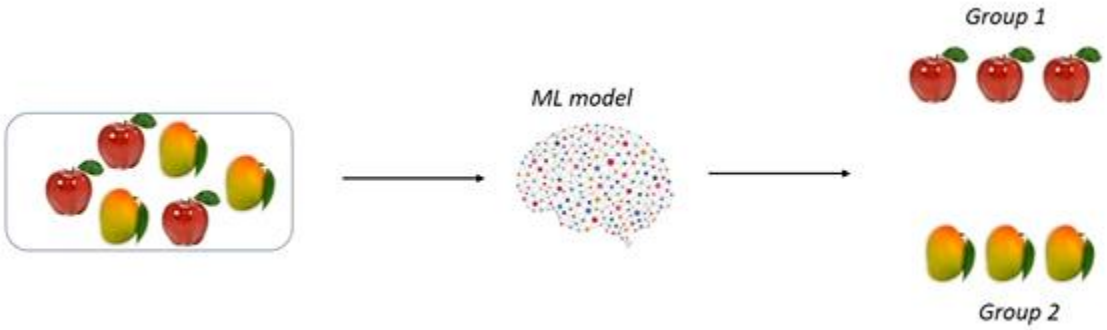
Types of Machine Learning



Supervised Learning: In supervised learning, the machine learning algorithm learns from **labelled data,** where each input (e.g., an image) is paired with a correct output (e.g., a label like "cat" or "dog"). The model learns to identify patterns to make accurate predictions on new, unseen data.

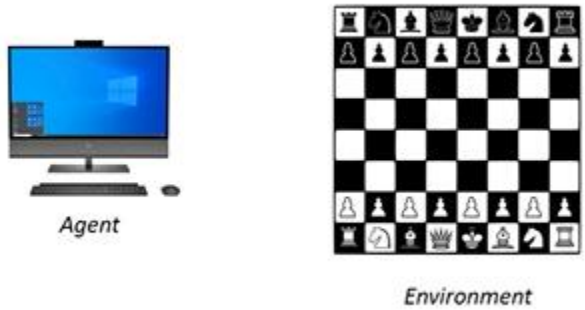


Unsupervised Learning: In unsupervised learning, the machine learning algorithm learns from **unlabelled data,** Here we provide several images to our machine learning models and we do not tell the label of image then model tries to find out the pattern of given images and group the images in different groups based on the pattern of the given images. Here, we are not telling the machine about label.



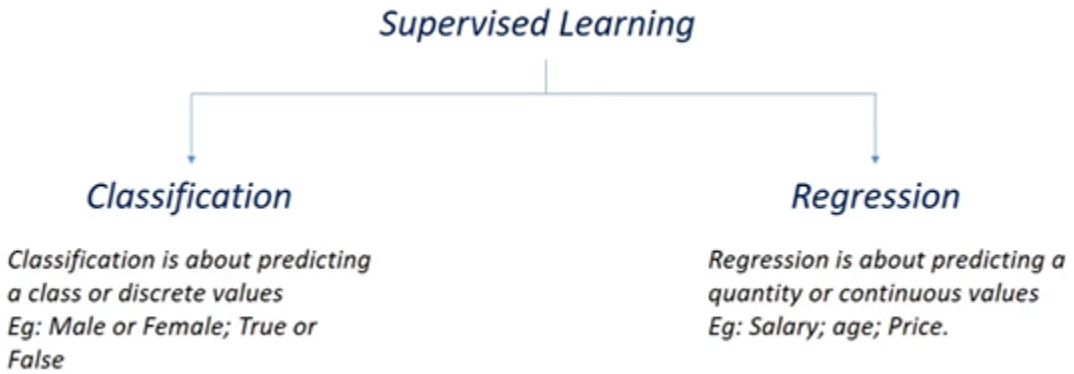
Reinforcement Learning: It is an area of Machine Learning concerned with how intelligent agents take actions in an environment to maximize its rewards. There are four main aspects of Reinforcement Learning such as **Environment, Agent, Action, Reward**.

Examples autonomous car, playing games like chess, automatic drones are based on reinforcement learning.

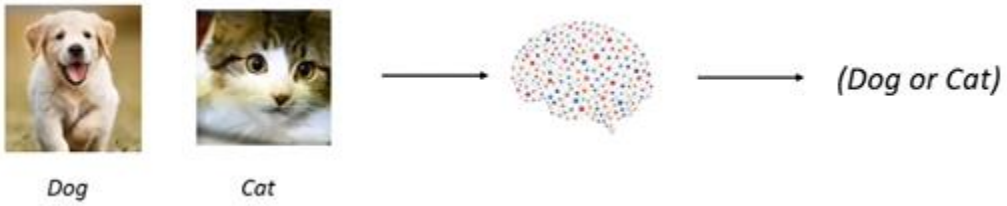


For each move (action) the agent gets rewards (positive or negative) based on bad move or true move for winning the game. Here, the machine or agent tries to learn how to play the games.

Types of Supervised Learning:

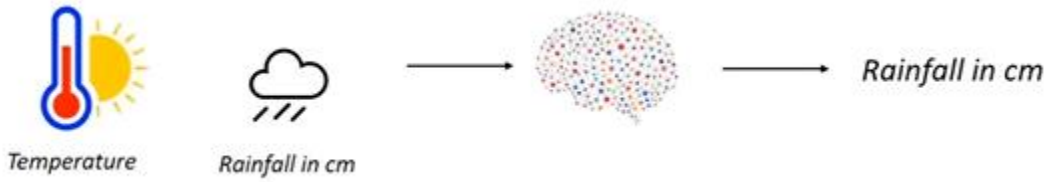


Classification: For Categorical value



Here, We are classifying the image whether it is dog or cat. In this case, we try to find the class.

Regression: For Continuous value



Here, we try to predict the rainfall in cm based on temperature. In this case, we try to find a number.

Several algorithms used for classification and regression:

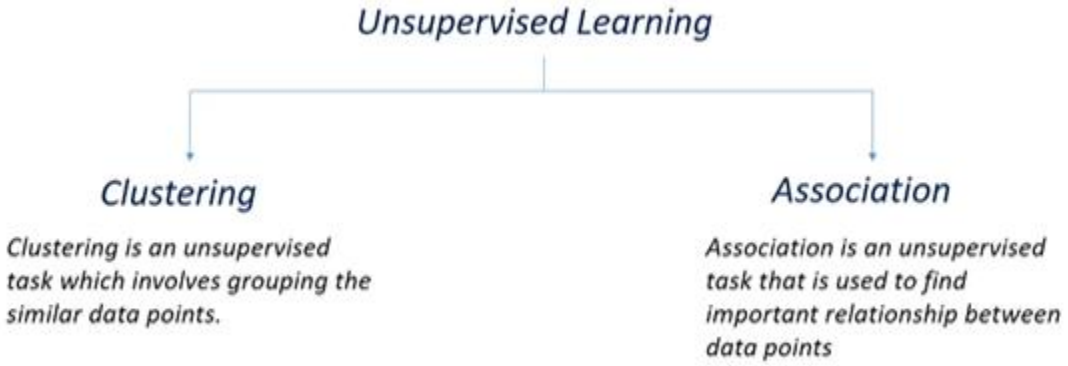
Classification:

- Decision Tree Classification
- Random Forest Classification
- K-nearest Neighbor

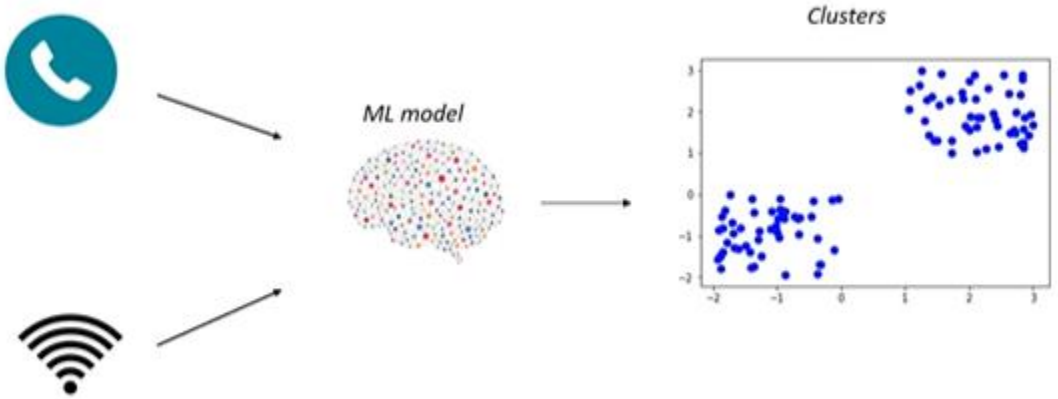
Regression:

- Logistic Regression
- Polynomial Regression
- Support Vector Machines

Types of Unsupervised Learning

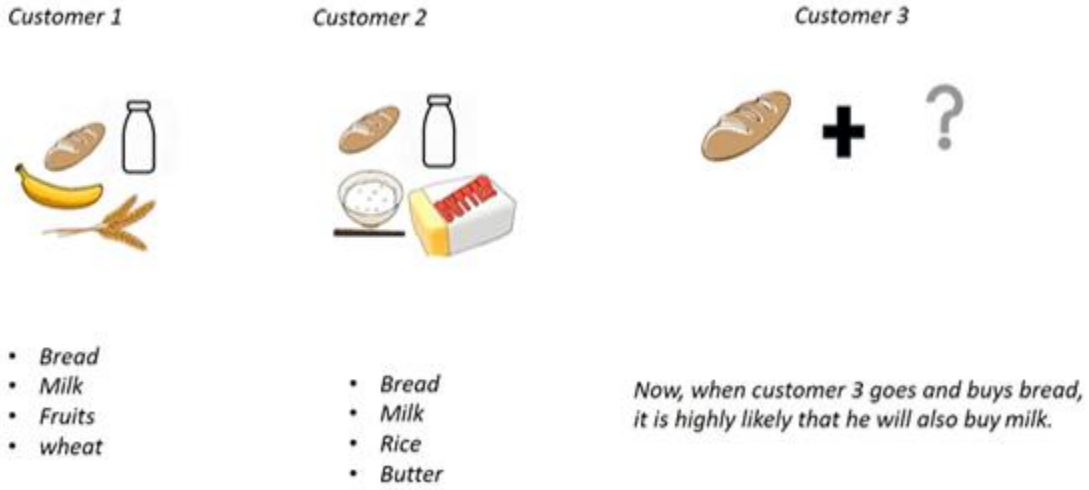


Clustering:



Suppose here we are giving offer for internet usage on phone call. We can cluster/group customer based on internet usage with phone call. (High internet usage customer can have more phone calls).

Association:



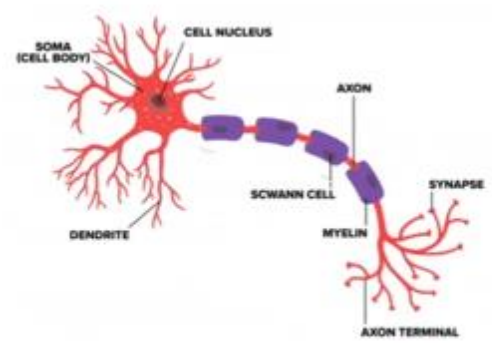
Here, we try to find the relation of customer buying different items. We know the customer who buys bread can also buy milk so for next customer who buys bread is highly likely to buy milk too.

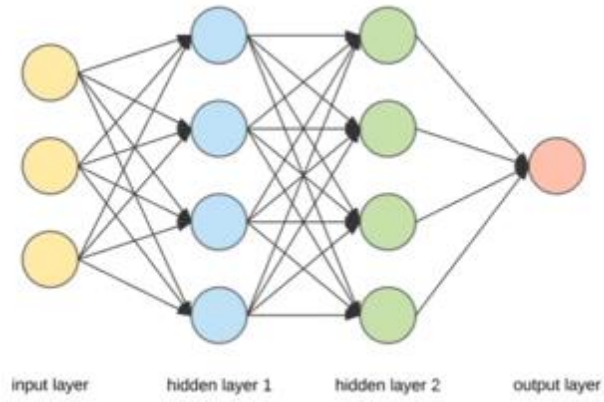
Several unsupervised learning algorithms:

- K-Means Clustering
- Hierarchical Clustering

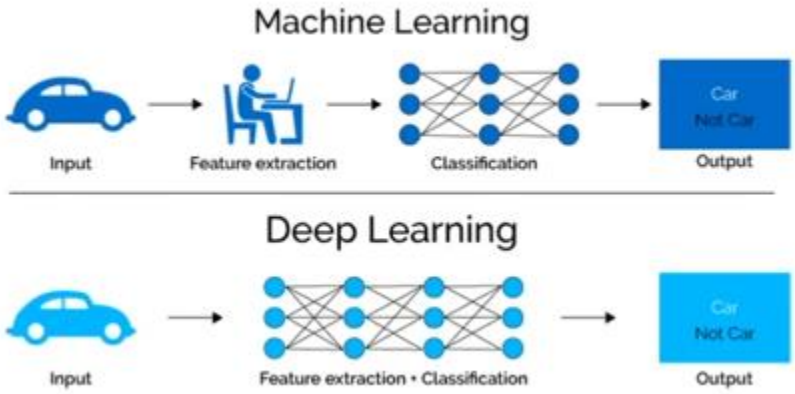
- Principal Component Analysis (PCA)
- Apriori

Deep Learning: It is the subfield of Machine Learning that uses Artificial Neural Networks to learn from the data. Example In human information passed with the help of neuron.





Machine Learning VS Deep Learning



In Machine Learning we manually tell the model about the features but in **Deep Learning model tries to identify the features on its own**, we do not need to manually provide the features. Deep Learning models are more powerful so that these model extract features by themselves.

Applications of Deep Learning are Healthcare System, Autonomous Cars, Computer Vision (eg. Face Recognition system), Natural Language Processing (eg. Chatbots like google assistant, siri).

Data Science (DS):

- Part of anything or the science of analysing and extracting information from large sets of data, which typically combines elements of statistics, mathematics, computing and other subjects.
- Multi-disciplinary fields of using scientific methods, algorithms, systems to extract knowledge and insights from structured and unstructured data.
- In short, Data science is the study of data that helps us derive useful insight for business decision making.
- Data Science is all about using tools, techniques, and creativity to uncover insights hidden within data.
- Data Science processes the raw data and solve business problems and even make prediction about the future trend or requirement.
- In short, data science empowers the industries to make smarter, faster, and more informed decisions.

Data science involves:

- Data Collection: Gathering raw data from various sources, such as databases, sensors, or user interactions.
- Data Cleaning: Ensuring the data is accurate, complete, and ready for analysis.
- Data Analysis: Applying statistical and computational methods to identify patterns, trends, or relationships.
- Data Visualization: Creating charts, graphs, and dashboards to present findings clearly.
- Decision-Making: Using insights to inform strategies, create solutions, or predict outcomes.

Real World Application of Machine Learning:

Machine Learning (ML) is widely used across industries to automate decisions, discover patterns, and make accurate predictions from data.

1. **Healthcare**

Applications

i. Disease prediction (diabetes, heart disease, cancer)

ii. Medical image analysis (X-ray, MRI, CT scan)
iii. Drug discovery and personalized treatment
iv. Patient risk monitoring

Examples

a. AI detecting cancer from X-rays
b. Predicting patient readmission risk

2. Finance & Banking

Applications

- Fraud detection
- Credit scoring and loan approval
- Stock market prediction
- Risk assessment

Examples

- Credit card fraud alerts
- Automated loan approval systems

3. E-Commerce & Retail

Applications

- Recommendation systems
- Customer behavior analysis
- Dynamic pricing
- Demand forecasting

Examples

- Amazon product recommendations
- Flipkart price optimization

4. Transportation & Autonomous Vehicles

Applications

- Self-driving cars
- Traffic prediction
- Route optimization

- Driver assistance systems

Examples

- Tesla Autopilot
- Google Maps traffic prediction

5. Social Media & Entertainment

Applications

- Content recommendation
- Sentiment analysis
- Fake news detection
- Image and video tagging

Examples

- YouTube video suggestions
- Facebook friend recommendations

6. Education

Applications

- Personalized learning paths
- Student performance prediction
- Automated grading
- Intelligent tutoring systems

Examples

- Coursera personalized courses
- AI-based exam evaluation

7. Manufacturing & Industry

Applications

- Predictive maintenance
- Quality control
- Supply chain optimization
- Fault detection

Examples

- Predicting machine failure before breakdown
- Automated defect detection

8. Agriculture

Applications

- Crop yield prediction
- Disease detection in plants
- Soil quality analysis
- Smart irrigation

Examples

- AI detecting crop diseases from images
- Weather-based yield forecasting

9. Cybersecurity

Applications

- Intrusion detection → **Intrusion Detection** is the process of monitoring network or system activities to detect **unauthorized access, attacks, or policy violations**.
- Malware and spam detection
- Network anomaly detection (Outlier Detection) → technique used to detect **cyber attacks, intrusions, or abnormal activities** in a computer network.

Examples

- Gmail spam filtering
- AI-based firewall systems

10. Natural Language & Speech Processing

Applications

- Speech recognition
- Language translation
- Chatbots and virtual assistants
- Text summarization

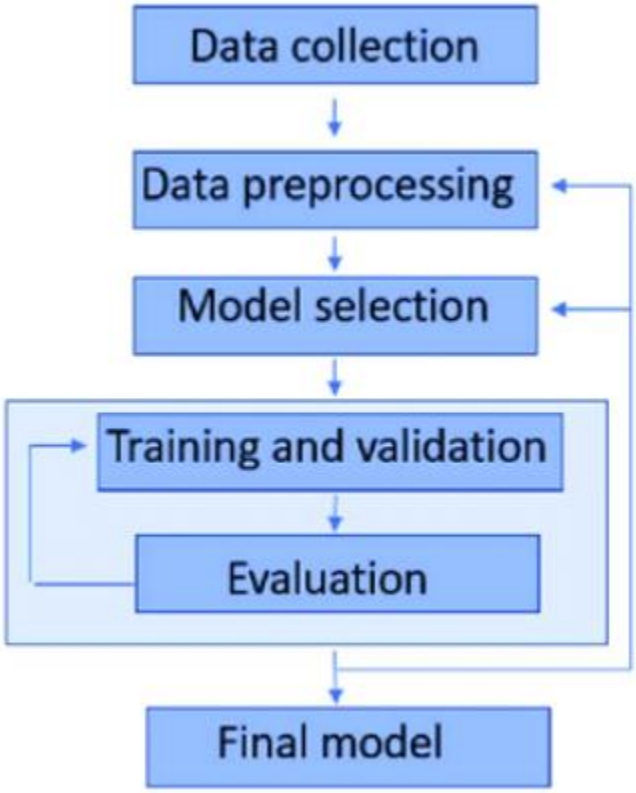
Examples

- Google Translate

- Siri, Alexa, ChatGPT

Machine Learning Workflow (Step-by-Step)

The Machine Learning workflow is a structured process that converts a real-world problem into a trained and deployable ML model.



1. Problem Definition

What are we solving?

- Classification (spam / not spam)
- Regression (house price prediction)
- Clustering (customer segmentation)

Example:
Predict whether a news article is *fake or real*.

2. **Data Collection**

Where does data come from?

- CSV files, databases
- APIs, web scraping
- Sensors, surveys

Example:
News articles with labels (real / fake).

How/where to find Machine Learning Data?

There are many places where we can find machine learning data.

- Kaggle → *kaggle.com*
- UCI Machine Learning Repository → *https://archive.ics.uci.edu/*
- Google Dataset Search → *https://datasetsearch.research.google.com/*

3. **Data Understanding & Exploration (EDA)**

Understand the data

- Data size, features, labels
- Missing values
- Data distribution & patterns

Tools: pandas, matplotlib, seaborn

4. **Data Preprocessing**

Clean and prepare data

- Handle missing values
- Encode categorical data (converting **non-numeric (text/label) data into numeric format** so that machine learning algorithms can understand and process it).
- Feature scaling (Standardization / Normalization) → **Feature Scaling** means adjusting the range of numerical features so that they are on a similar scale.
- Text cleaning (tokenization, stopword removal)

Example:
Convert text → numerical features using **TF-IDF**.

5. **Feature Engineering**

Improve model performance

- Feature selection
- Feature extraction → selecting or transforming raw data into important and meaningful features that help a machine learning model perform better.
- Create new features

Example:
Article length, number of keywords, sentiment score.

6. **Split Data**

Avoid overfitting

- Training set (70–80%)
- Testing set (20–30%)
- Validation set (optional)

7. **Model Selection**

Model Selection in Machine Learning is the process of choosing the best suited model for a particular problem. Selecting a model depends on various factors such as the dataset, task, nature of the model etc.

Choose algorithm

- Linear Regression → prediction
- Logistic Regression → classification
- Decision Tree / Random Forest → structured data→ **organized in a fixed format**, usually in rows and columns.
- SVM → high-dimensional data→ data that has a **large number of features (variables/columns)**.
- Neural Networks → complex patterns → used for solving **complex pattern recognition and prediction problems**.

Neural networks can be used in:

Image Processing (Computer Vision)

- Image classification
- Object detection
- Face recognition
- Medical image diagnosis →Detecting tumors in MRI scans
- Self-driving car

Natural Language Processing (NLP)

- Sentiment analysis
- Chatbots
- Language translation
- Text summarization

Examples:

- Google Translate
- ChatGPT

Healthcare Applications

- Disease prediction
- Cancer detection
- ECG signal classification
- Radiology report generation
- Drug discovery etc.

How to choose the right Machine Learning Model?

Choosing the right ML model depends on problem type, data, size, complexity, and constraints.

Identify the Problem Type

First, clearly define **what you want to predict**.

Problem	Model Choices
Continuous value	Linear / Ridge / Lasso Regression
Binary classification	Logistic Regression, SVM, Decision Tree
Multi-class classification	Random Forest, XGBoost, Neural Network
No labels	K-Means, Hierarchical Clustering
Sequential data	HMM, LSTM, Transformers

Understand Your Data Type

Data Type	Best Models
Tabular (CSV, Excel)	Linear Models, Tree-based models
Text (NLP)	Naive Bayes, SVM, TF-IDF + LR, Transformers
Images/Videos	CNN
Time-series	ARIMA, LSTM

Dataset Size Matters

Data Size	Recommended Models
Small data	Linear/Logistic Regression, Naive Bayes
Medium data	Random Forest, SVM
Large data	XGBoost, Deep Learning

Rule: **Start simple, then increase complexity.**

Speed & Resources

Constraint	Model
Low compute	Linear, Naive Bayes
Real-time prediction	Logistic Regression
GPU available	Deep Learning

Based on the task we need to carry out

- Classification tasks →SVM, Logistic Regression, Decision trees etc.
- Regression tasks → Linear regression, SVM, Random Forest, Polynomial regression etc.
- Clustering tasks → K-means clustering, Hierarchical Clustering

Real-World Example

Problem: Fake News Detection

- Data: Text
- Size: Medium
- Model Choice:
    - Baseline: Naive Bayes
    - Improved: Logistic Regression + TF-IDF
    - Advanced: BERT (**Bidirectional Encoder Representations from Transformers**)

Quick Decision

If you have...	**Choose**
Simple data	Linear / Logistic
Mixed features	Random Forest
High-dimensional data	SVM

If you have...	Choose
Images/Videos	CNN
Text/Speech data	RNN/LSTM, NB / Transformers
Need explainability	Linear / Tree
Numerical data	SVM, Logistic, Decision trees etc.

8. **Model Training**
   - Model Learns patterns from data
   - **Model training** is the process of **teaching a machine learning or deep learning model** to make predictions or decisions based on data.
   - In simple terms, We feed the model input data and expected outputs (labels), and the model **learns patterns** from the data.

9. **Model Evaluation**

Measure performance

- **Classification:** Accuracy, Precision, Recall, F1-score
- **Regression:** MAE, MSE, RMSE, R^2
- Confusion Matrix

10. **Model Deployment**

Make model usable

- Flask / FastAPI
- Cloud (AWS, GCP)
- Mobile / Web apps

Example:
Fake news detection API.

11. **Monitoring & Maintenance**

Keep model updated

- Monitor accuracy drift
- Retrain with new data
- Handle data changes

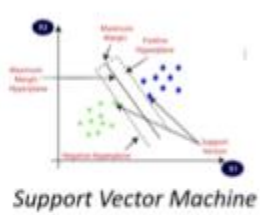
Cross Validation

Cross Validation is a resampling technique used to evaluate machine learning models by partitioning data into multiple folds, training and testing the model repeatedly, and averaging the results to estimate generalization performance.

Example: Let's say we have two models such as Support Vector Machine and Logistic Regression then we want to find which model is best suited for a particular problem. In that case we use a process to evaluate our model called Cross Validation.

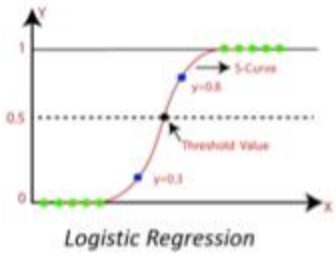
We can find cross validation library in **sklearn** library in python.

In this case we try to find the best model by comparing accuracy score of our models.



	Dataset					Accuracy
Iteration 1	Train	Train	Train	Train	Test	88%
Iteration 2	Train	Train	Train	Test	Train	83%
Iteration 3	Train	Train	Test	Train	Train	86%
Iteration 4	Train	Test	Train	Train	Train	81%
Iteration 5	Test	Train	Train	Train	Train	84%

$$\text{Mean Accuracy} = \frac{88 + 83 + 86 + 81 + 84}{5} = 84.4\ \%$$



	Dataset					Accuracy
Iteration 1	Train	Train	Train	Train	Test	90%
Iteration 2	Train	Train	Train	Test	Train	88%
Iteration 3	Train	Train	Test	Train	Train	86%
Iteration 4	Train	Test	Train	Train	Train	91%
Iteration 5	Test	Train	Train	Train	Train	85%

$$\text{Mean Accuracy} = \frac{90 + 88 + 86 + 91 + 85}{5} = 88\ \%$$

Now, let's compare the accuracy of SVM and Logistic Regression model, here we find the accuracy score is more for Logistic Regression.

Accuracy score for SVM = 84.4 %

Accuracy score for Logistic Regression = 88 %

In this case, the Logistic Regression model is best for this particular problem and this depends on the type of dataset we have.

Types of Cross Validation

1. Hold-Out Validation

- One train–test split
- Fast but less reliable

2. K-Fold Cross Validation

- Balanced and widely used
- Split data into **k equal parts (folds)**
- K = 5 or 10 is common
- 5-Fold CV →5 different train-test combinations

3. Stratified K-Fold

- Maintains class distribution
- Best for **imbalanced datasets**

Example: Fraud detection

4. Leave-One-Out (LOOCV)

- Each sample is a test set once
- Special case of k-fold where k = number of data points
- Train on all data except 1 → test on that 1
- Repeat for all points
- Accurate but **computationally expensive**
- Very accurate, very slow

5. Time Series Cross Validation

- Respects time order
- No shuffling

Example: Stock price prediction

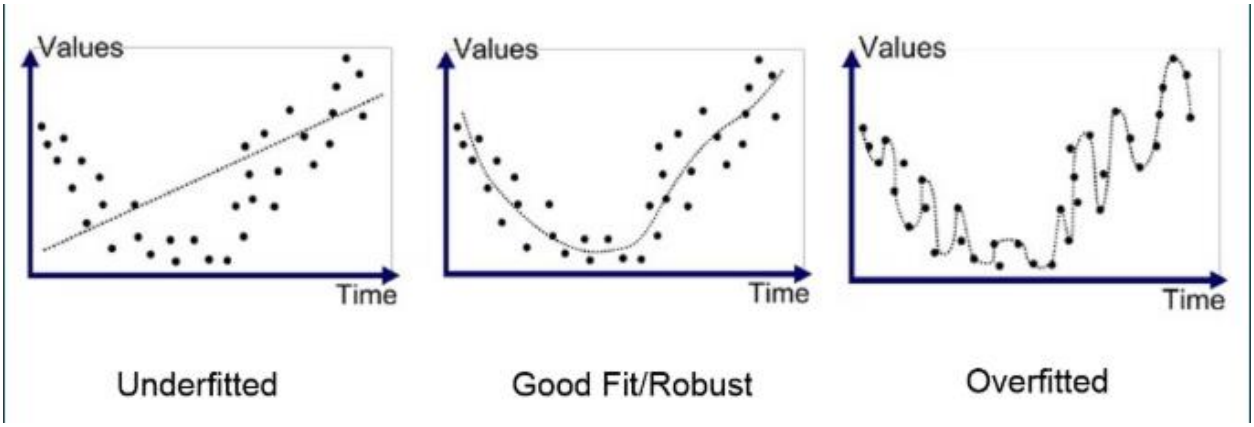
You can learn more about cross validation from here: *https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.cross_val_score.html*

Overfitting Vs Underfitting

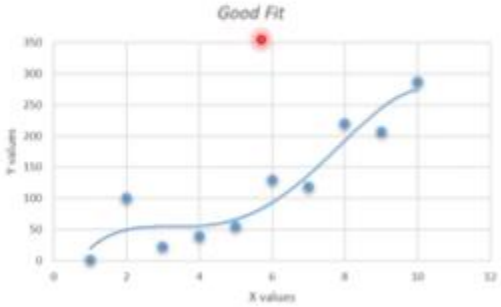
Overfitting

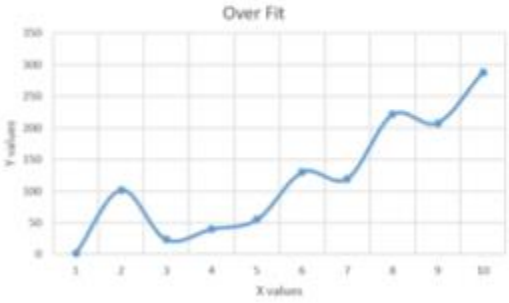
Overfitting refers to a model that models the training data too well. Overfittings when a model learns the detail and noise in the training dataset to the detent that it negatively impacts the performance of the model.

Overfitting occurs when a machine learning model learns the training data too well, including noise and irrelevant patterns, and therefore performs poorly on unseen (test) data.



X	1	2	3	4	5	6	7	8	9	10
Y	1.38	101.41	23.34	39.89	55.23	129.91	119.33	221.09	207.43	287.80





Real-life example:
A student memorizes answers for one question paper but fails in a new exam.

Student Memorizing Answers

- Student memorizes exact questions from last year
- Exam questions change slightly
- Student fails

Model learned noise instead of concepts

Driving Test on One Road

- Learns to drive only on **one route**
- Performs perfectly on that road
- Fails on new roads

Too specific learning

Spam Filter Example

- Trained only on emails with word **"FREE!!!"**
- Detects those perfectly
- Misses spam using different words

Learns limited patterns

Face Recognition with Few Photos

- System trained with only one photo per person
- Works for same photo
- Fails under different lighting or angles

High training accuracy, low real-world accuracy

How to Identify Overfitting

Indicator	Observation
Training accuracy	Very high
Testing accuracy	Low
Error gap	Large difference between train & test

Overfitting vs Underfitting

Aspect	Underfitting	Overfitting
Model complexity	Too simple	Too complex
Training error	High	Very low
Testing error	High	High
Generalization	Poor	Poor

Why Overfitting Happens

- Very complex model
- Small dataset
- Too many features
- No regularization
- Training too long (deep learning)
- More number of layers in Neural Network

Techniques to Prevent Overfitting

1. Train with More Data

- More data → better generalization

2. Feature Selection

- Remove irrelevant or redundant features

3. Regularization

Penalizes large weights→ **adding a penalty term to the model's loss function** that discourages the model from assigning very high values to its parameters (weights).

Type	Use
L1 (Lasso)	Feature selection
L2 (Ridge)	Weight control

4. Cross Validation

- Detects overfitting early
- Helps choose correct model complexity

5. Early Stopping

- Stop training when validation error increases

6. Pruning (Tree Models)

- Limit tree depth
- Reduce number of nodes/layers in the neural network

7. Ensemble Methods

→ Techniques in machine learning where multiple models (often called "weak learners") are combined to create a stronger, more accurate model.

- Random Forest
- Bagging, Boosting

8. Dropout (Deep Learning)

- Randomly drops neurons during training

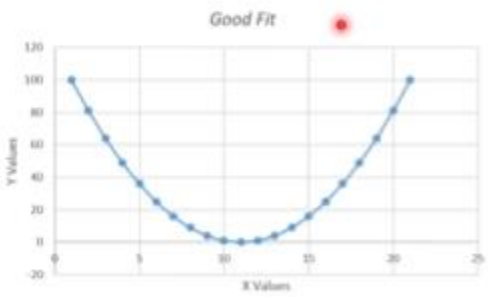
In short, Overfitting is a condition where a machine learning model performs exceptionally well on training data but poorly on unseen data due to learning noise and complex patterns.

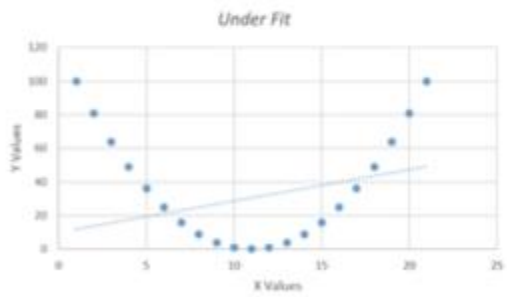
Underfitting

Underfitting occurs when a machine learning model is too simple to capture the underlying patterns in the data.
As a result, it performs poorly on both training and test data.

The model **has not learned enough** from the data.

X	-10	-9	-8	-7	.....	0	.....	7	8	9	10
Y	100	81	64	49	.....	0	.....	49	64	81	100





How to Identify Underfitting

Indicator	**Observation**
Training accuracy	Low
Testing accuracy	Low
Error	High on both sets

Underfitting vs Overfitting

Aspect	**Underfitting**	**Overfitting**
Model complexity	Too simple	Too complex
Training error	High	Very low

Aspect	Underfitting	Overfitting
Testing error	High	High
Bias	High	Low
Variance	Low	High

Why Underfitting Happens

- Model is too simple
- Important features missing
- Too much regularization
- Insufficient training time
- Poor feature representation
- **Choosing a wrong model**
- Less complexity of the model
- Less variance but high bias

How to Fix Underfitting

1. Choosing the correct model appropriate for the problem

2. Increase Model Complexity

- Linear → Polynomial
- Shallow tree → Deeper tree

3. Add More Relevant Features

- Feature engineering
- Better feature extraction

4. Reduce Regularization

- Lower L1/L2 penalty

5. Train Longer

- More epochs (DL)
- Better convergence

6. Use More Powerful Models

- Random Forest
- Gradient Boosting
- Neural Networks

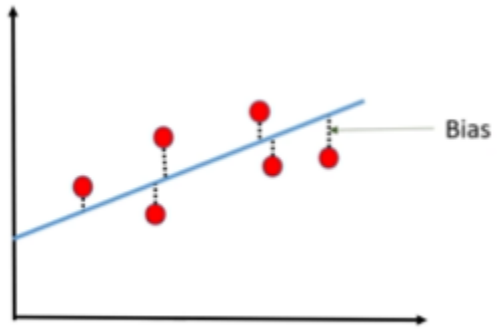
In short, Underfitting is a condition where a machine learning model fails to capture the underlying data patterns due to insufficient complexity, resulting in poor performance on both training and test datasets.

Bias–Variance Tradeoff

This is a Machine learning model optimization techniques.

Situation	**Bias**	**Variance**
Underfitting	High	Low
Overfitting	Low	**High**

Bias: It is the difference between the average prediction of our model and the correct value which we are trying to predict.



How to Reduce Bias

- Increase model complexity
- Add relevant features
- Reduce regularization

Variance: Variance is the amount that the estimate of the target function will change if different training data was used.

How to Reduce Variance

- Use more training data
- Apply regularization (L1/L2)
- Use ensemble methods
- Reduce model complexity

Point to Know:

- High Bias → underfitting
- High Variance → overfitting
- Balance is essential for good ML models

Techniques to have better Bias – Variance Tradeoff:

- Good model selection
- Regularization
- Dimensionality reduction
- Ensemble methods

Loss Function

A loss function in machine learning is a mathematical function that measures how well a model's predictions match the actual target values.

Loss function measures how far an estimated value is from its true value.

Lower loss → better model performance on the training data.

Common Types of Loss Functions

For Regression (predicting continuous values)

1. **Mean Squared Error (MSE)**

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

Here, n is the total number of observation.
Here, we are squaring the error to neglect the negative value.

- Penalizes larger errors more heavily (squared term).

2. **Mean Absolute Error (MAE)**

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|$$

- Less sensitive to outliers than MSE.
- Measures average magnitude of errors.

For Classification (predicting categories)

1. **Binary Cross-Entropy (Log Loss) – for 2 classes**

$$\text{Loss} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

- Penalizes incorrect predictions more when the model is confident and wrong.

2. **Categorical Cross-Entropy – for multiple classes**

$$\text{Loss} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$$

- Generalization of binary cross-entropy for multi-class classification.

Choosing the right loss function depends on:

- Type of problem (regression vs classification)
- Presence of outliers

Work Flow of a Machine Learning Project

- Data collection
- Data pre-processing
- Data analysis
- Train Test split
- Build model
- Evaluation

Evaluation metric for Classification: Accuracy score

Evaluation metric for Regression: Mean Absolute Error, Mean Squared Error

In classification, Accuracy score is the ratio of number of correct predictions to the total number of input data points.

Accuracy score is calculated as:

$$\text{Accuracy Score} = \frac{\text{Number of correct predictions}}{\text{Total Number of data points}} \times 100\ \%$$

We have python library from sklearn to calculate accuracy_score. To use this we have import it first.

```
from sklearn.metrics import accuracy_score
```

Mean Squared Error measures the average of the squares of the errors, that is the average squared difference between the estimated values and the actual value.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

Low MSE value represents we have very good model.

We have python library from sklearn to calculate mean squared error. To use this we have import it first.

```
from sklearn.metrics import mean_squared_error
```

Model Parameters

Model parameters are the internal parameters/variables of a model that are **learned automatically from the training data** during the training process.

Key Characteristics

- Learned during training
- Updated by optimization algorithms (e.g., Gradient Descent)
- Define how the model makes predictions

Examples

- **Linear Regression**:
  - Weights (w) and bias (b)
- **Logistic Regression**:
  - Weights and bias
- **Neural Networks**:
  - Weights and biases of each layer
- **Support Vector Machine (SVM)**:
  - Support vectors and coefficients

Example

Equation for linear regression:

$$y = wx + b$$

For multiple input features and weights $Y = w_1x_1 + w_2x_2 + w_3x_3 + b$

- $w, b \rightarrow$ **model parameters**
- $x - feature\ or\ input\ variable$
- y- target or output variable

Weight decides how much influence the input will have on the output.

Bias is the offset value given to the model. Bias is used to shift the model in a particular direction. It is similar to Y-intercept. 'b' is equal to 'Y' when all the feature values i.e x are zero.

Hyperparameters

Hyperparameters are **set by the user before training starts** and control how the learning process happens. The model does **not learn** them automatically. These are adjustable parameters used to obtain an optimal model and these are external parameters.

Key Characteristics

- Set manually (or via tuning techniques)
- Control model complexity and training behavior
- Strongly affect performance and overfitting

Examples

- Learning rate
- Number of epochs
- Batch size
- Number of hidden layers
- Number of neurons per layer
- Regularization parameter (λ)
- k in k-NN
- Depth of decision tree

Summary

- **Parameters** are learned from data and directly used in prediction.

- **Hyperparameters** guide the training process and model structure.
- Good performance depends on **both** well-learned parameters and well-chosen hyperparameters.

Learning Rate: It is a tuning parameter in an optimization algorithm that determines the step size at each iteration while moving toward a minimum of a loss function.

Number of Epochs: It represents the number of times the model iterates over the entire dataset.

Loss Function: Loss function measures how far an estimated value is from its true value. It is helpful to determine which model performs better and which parameters are better.

$$Loss = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

Where, Y_i = True value

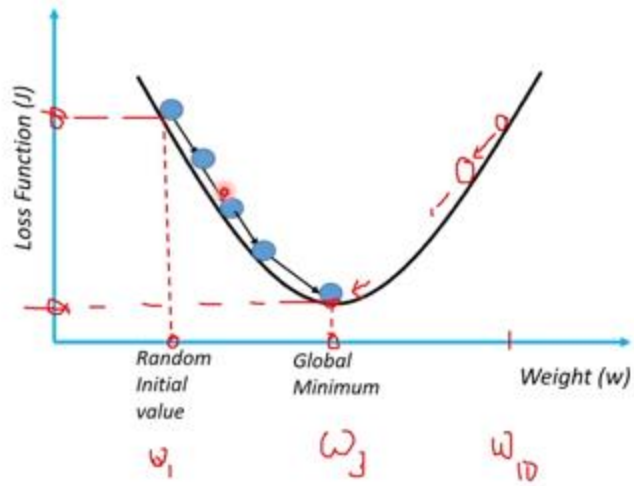
Y_i^ = Predicted value

→If the loss function value is high then the model is not performing better.

Model Optimization: Optimization refers to determining best parameters for a model, such that the loss function of the model decreases, as a result of which the model predict more accurately.

Gradient Descent: It is the model optimization technique. When we plot weight and loss function then for weight w_1/$w_{10,}$ it has higher loss value. On decreasing loss function closer to zero we will have minimum loss function which is at point $w_{3.}$ This point is called global minimum.

We know that the loss function is the difference between true and predicted value. So, if we plot the graph between weight and loss function value then we will get U-shaped curved then this U-shaped curved is called gradient descent.



Gradient descent in 2D

Gradient Descent is an optimization algorithm used for minimizing the loss function in various machine learning algorithms. It is used for updating the parameters of the learning model.

$$w = w - L*dw$$

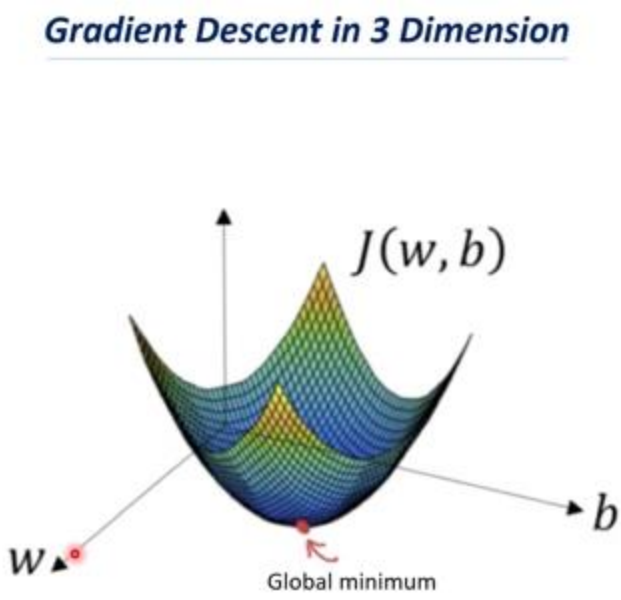
$$b = b - L*db$$

w --> weight
b --> bias
L --> Learning Rate

dw --> Partial Derivative of loss function with respect to ~~m~~ w

db --> Partial Derivative of loss function with respect to ~~c~~ b

For updating weight and bias we will use above formula.



The main job of gradient descent is to find the best parameter that the model can have so that model can have minimum loss function then we can make better/accurate prediction.